

Práctico 1

Comandos, redirección, pipes, señales y llamadas al sistema

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento presenta la resolución integral del Práctico 1 de la materia Sistemas Operativos (UNRC, 2026). Cada ejercicio se justifica con la teoría de los capítulos 1, 2 y 3 de las *Notas del curso* (Marcelo Arroyo) y del capítulo 1 de *Modern Operating Systems* (Tannenbaum & Bos). El objetivo es transparentar el modelo `fork()/exec()/wait()` junto con los mecanismos de redirección, pipes, señales y llamadas al sistema que provee la API POSIX. Todo el código fue compilado con `gcc -Wall -Wextra` y verificado en una distribución tipo UNIX.

Índice

1. Introducción	2
2. Ejercicio 1 — Comandos del shell	2
3. Ejercicio 2 — Redirección, secuencia y filtrado	5
4. Ejercicio 3 — Pipes, sin temporales	6
5. Ejercicio 4 — Shell scripts y permisos	6
6. Ejercicio 5 — Programa C con <code>exit status</code>	7
7. Ejercicio 6 — Versión propia de <code>system()</code>	8
8. Ejercicio 7 — Comunicación padre/hijo por <code>pipe()</code>	9
9. Ejercicio 8 — Comunicación por FIFO	9
10. Ejercicio 9 — Mini shell <code>minish</code>	10
11. Ejercicio 10 — Captura de SIGINT	11
12. Ejercicio 11 — Mínimo global con p hijos y un pipe	12
13. Ejercicio 12 — Lectura con <code>timeout</code> usando <code>alarm</code>	13
Cierre	14
Referencias	14

1. Introducción

Un **sistema operativo (SO)** es la capa de software que abstrae el hardware, administra los recursos disponibles (CPU, memoria, dispositivos de E/S) y ofrece servicios a las aplicaciones mediante *llamadas al sistema*. Tannenbaum y Bos, en el capítulo 1 de *Modern Operating Systems*, lo presentan a la vez como *máquina extendida*—que oculta la fealdad del hardware tras abstracciones uniformes—y como *administrador de recursos*, que multiplexa CPU y memoria en tiempo y en espacio entre múltiples programas.

Las Notas del curso enfatizan tres elementos transversales a este práctico:

- **Modo kernel y modo usuario.** La CPU ejecuta el código del SO con privilegios totales (modo supervisor) y los procesos de usuario en modo restringido. El cambio se realiza vía *traps* (syscalls) o interrupciones.
- **Proceso.** Instancia de un programa en ejecución, con espacio de direcciones propio (código, datos globales, pila), descriptor en el kernel (PCB) y archivos abiertos heredados.
- **Multitarea / multiprogramación.** El SO mantiene varios procesos en memoria y multiplexa la CPU mediante *context switches* (pre-emptive en sistemas de tiempo compartido).

Sobre estas bases, el shell se entiende como un intérprete REPL que compone procesos mediante operadores de redirección (<, >, »), secuencia (;), condicionales (&&, ||), pipes (|) y ejecución concurrente (&). Cada operador encuentra una traducción directa en combinaciones de las syscalls `fork()`, `exec()`, `wait()`, `pipe()`, `dup2()` y `open/close/read/write`.

2. Ejercicio 1 — Comandos del shell

1.a Procesos en ejecución y PID del primer proceso

```
ps -e -o pid,ppid,state,comm | head -n 20
```

`ps -e` lista todos los procesos del sistema. La opción `-o` permite seleccionar las columnas relevantes (PID, PPID, estado y nombre). El proceso con **PID 1** es el primer proceso lanzado por el kernel apenas concluye la inicialización del SO; en Linux suele ser `systemd` (o `init` en variantes clásicas), en macOS `launchd`. Las Notas del curso lo describen como “el primer proceso de usuario preconfigurado” que se lanza al final de la rutina de arranque.

1.b Usuarios conectados y usuario actual

```
who
who am i
whoami
```

`who` lista todas las sesiones abiertas, `who am i` muestra únicamente la asociada a la terminal actual y `whoami` imprime el *UID efectivo* del proceso shell. La diferencia ilustra la naturaleza multiusuario de UNIX: el SO mantiene una tabla de sesiones (`utmp/wtmp`) que asocia terminales con usuarios autenticados.

1.c Directorio corriente

```
pwd
echo "$PWD"
```

`pwd` consulta el directorio asociado al proceso shell, mientras que `$PWD` es una variable de ambiente que el shell mantiene sincronizada. Ambos comandos cumplen el requisito del enunciado de usar *al menos dos formas distintas*.

1.d Directorio home del usuario

```
echo "$HOME"
cd ~
pwd
```

`HOME` es una variable de ambiente fijada en el momento del *login* a partir de `/etc/passwd`. El símbolo `~` es un atajo que el shell expande a `$HOME` antes de ejecutar el comando.

1.e Metadatos de archivos

```
ls -la .
ls -la /
stat README.md /
```

`ls -la` muestra entradas ocultas y metadatos básicos (permisos, dueño, grupo, tamaño, fecha). `stat` agrega información de bajo nivel: inodo, cantidad de hard links, timestamps y dispositivo contenedor. Recordemos que en UNIX un archivo es una secuencia de bytes identificada por un inodo y la entrada de directorio asocia un nombre con ese inodo.

1.f Listar contenido sin editor

```
cat README.md
more README.md
less README.md
```

`cat` vuelca todo el archivo a `stdout` mediante `read()` y `write()`. `more` y `less` agregan una capa de paginación; `less` es más flexible al permitir desplazamiento bidireccional.

1.g Variables de ambiente

```
env | sort
```

Las variables de ambiente forman parte del *contexto* del proceso y se pasan a los hijos mediante el tercer parámetro de `main` (`char *envv[]`). Las más relevantes son `USER`, `HOME`, `PWD`, `SHELL`, `PATH` y `LANG`.

1.h Directorios donde el shell busca comandos

```
echo "$PATH" | tr ':' '\n'
```

`PATH` es una lista de directorios separados por dos puntos. Al recibir un comando sin barras, el shell recorre esos directorios en orden y ejecuta el primer archivo regular con permiso de ejecución que coincida. Por seguridad, el directorio actual `.` no suele estar incluido.

1.i Ubicación de un comando con which

```
which cat
```

which reporta el *full path* del primer ejecutable que el shell encontrará para ese nombre según el PATH vigente.

1.j Crear un archivo con al menos tres comandos

```
touch archivo.txt
echo "Primera linea" > archivo.txt
date "+%Y-%m-%d %H:%M:%S" >> archivo.txt
echo "Tercera linea" >> archivo.txt
cat archivo.txt
```

touch crea (o actualiza) el archivo. El operador > trunca y reescribe; » abre en modo *append*. Internamente el shell, antes de `exec`, abre el archivo y reasigna el descriptor 1 con `dup2()`.

1.k Crear un enlace simbólico

```
ln -s archivo.txt archivo_link.txt
ls -li archivo.txt archivo_link.txt
```

Un enlace simbólico es un archivo cuyo contenido es la *ruta* del destino. Tiene su propio inodo y `ls -li` muestra la flecha -> hacia el archivo real.

1.l Eliminar el enlace y comparar con hard link

```
rm archivo_link.txt
ln archivo.txt archivo_hard.txt
ls -li archivo.txt archivo_hard.txt
```

Eliminar el symlink *no* afecta al original: solo se borra la entrada de directorio que contenía la ruta. Un *hard link*, en cambio, crea otra entrada que comparte el mismo inodo: el archivo solo desaparece del disco cuando el contador `st_nlink` llega a cero.

Característica	Symbolic link	Hard link
Tipo de referencia	Por ruta (texto)	Por inodo
Inodo propio	Sí	No (comparte)
Sobrevive al rename del original	Solo si la ruta sigue válida	Sí
Sobrevive al borrado del original	No (queda <i>roto</i>)	Sí
Cruza filesystems	Sí	No
Permitido sobre directorios	Sí	Generalmente no

Cuadro 1: Comparación entre *symbolic links* y *hard links*.

1.m Programa C que muestra los argumentos

```
1 #include <stdio.h>
```

```

2
3 int main(int argc, char *argv[]) {
4     printf("argc = %d\n", argc);
5     for (int i = 0; i < argc; i++) {
6         printf("argv[%d] = %s\n", i, argv[i]);
7     }
8     return 0;
9 }

```

Compilación y ejecución:

```

gcc -Wall -Wextra -o hello hello.c
./hello arg1 arg2 arg3

```

Salida:

```

argc = 4
argv[0] = ./hello
argv[1] = arg1
argv[2] = arg2
argv[3] = arg3

```

¿Por qué `./hello arg1 ... argn`? Cuando el usuario escribe un nombre sin barras, el shell asume que se trata de un comando externo y lo busca *únicamente* en los directorios listados en PATH. El directorio actual `.` no suele estar incluido para evitar el ataque clásico de un binario malicioso colocado en `cwd`. El prefijo `./` indica una ruta relativa explícita y evita la búsqueda en PATH.

3. Ejercicio 2 — Redirección, secuencia y filtrado

Los archivos `correos1.txt` y `correos2.txt` contienen direcciones de correo (una por línea) con duplicados intencionales.

2.a Lista ordenada y sin duplicados con archivos temporales

```

cat correos1.txt correos2.txt > /tmp/correos_todos.txt
sort /tmp/correos_todos.txt > /tmp/correos_ordenados.txt
uniq /tmp/correos_ordenados.txt > correos_ordenados_sin_duplicados.txt

```

`cat` concatena ambos archivos; `sort` los ordena; `uniq` elimina duplicados *consecutivos*. La combinación `sort | uniq` elimina globalmente cualquier repetición. Los temporales se ubican en `/tmp` porque el SO los purga periódicamente.

2.b En una sola línea con operador secuencial ;

```

cat correos1.txt correos2.txt > /tmp/correos_todos.txt ; sort /tmp/correos_todos.txt > /tmp/correos_ordenados.txt ; uniq /tmp/correos_ordenados.txt > correos_ordenados_sin_duplicados.txt

```

El operador `;` ejecuta los comandos en secuencia *independientemente* del éxito o fracaso del anterior. Si quisiéramos abortar al primer error usaríamos `&&`, pero el enunciado pide explícitamente operador secuencial.

2.c Filtrar y contar direcciones de google.com

```
grep -E '@google\.com$' correos_ordenados_sin_duplicados.txt | wc -l
```

- `grep -E` interpreta el patrón como expresión regular extendida.
- `@google\.com$` fuerza coincidencia literal de `@google`, un punto literal (no un comodín), `com` y el ancla de fin de línea `$`.
- El pipe envía la salida de `grep` a `wc -l`, que cuenta líneas.

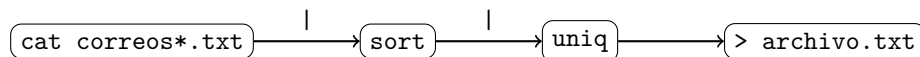
Con los archivos de ejemplo provistos el resultado esperado es 3.

4. Ejercicio 3 — Pipes, sin temporales

```
cat correos1.txt correos2.txt | sort | uniq > correos_ordenados_sin_duplicados.txt
```

Las Notas del curso describen al *pipe* como un buffer acotado en el kernel que implementa el patrón productor/consumidor. Cuando el shell ve `cmd1 | cmd2`:

1. Crea el pipe con `pipe(p)`.
2. Hace `fork()` para lanzar ambos procesos en paralelo.
3. Redirige `stdout` de `cmd1` al extremo de escritura (`p[1]`) y `stdin` de `cmd2` al extremo de lectura (`p[0]`) usando `dup2()`.
4. Cierra los descriptores que cada proceso no usa: si quedaran abiertos, el lector nunca vería EOF.



A diferencia del ejercicio 2, los datos viajan por buffers en memoria sin crear archivos en el disco, los tres procesos ejecutan concurrentemente y no hay limpieza posterior.

5. Ejercicio 4 — Shell scripts y permisos

4.1 Probar el script con sh

Archivo `procesar_correos_sin_interprete.sh` (sin shebang):

```
cat correos1.txt correos2.txt | sort | uniq > correos_ordenados_sin_duplicados.txt
echo "Archivo generado: correos_ordenados_sin_duplicados.txt"
```

Invocación:

```
sh procesar_correos_sin_interprete.sh
```

Como `sh` se invoca explícitamente, el archivo no necesita permiso de ejecución; basta con poder leerlo.

4.2 Agregar shebang y permisos

```
#!/bin/sh
set -eu
```

```
cat correos1.txt correos2.txt | sort | uniq > correos_ordenados_sin_duplicados.txt
echo "Archivo generado: correos_ordenados_sin_duplicados.txt"
```

```
chmod +x procesar_correos.sh
./procesar_correos.sh
```

El shebang `#!/bin/sh` le indica al kernel qué intérprete usar al invocar `execve()`. La opción `set -eu` aborta al primer error y trata como error variables sin definir; no es exigida por el enunciado pero es buena práctica defensiva.

6. Ejercicio 5 — Programa C con exit status

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(int argc, char *argv[]) {
6     char *endptr = NULL;
7     long value;
8
9     if (argc != 2) {
10         fprintf(stderr, "Uso: %s <0-255>\n", argv[0]);
11         return 2;
12     }
13
14     errno = 0;
15     value = strtol(argv[1], &endptr, 10);
16
17     if (errno != 0 || endptr == argv[1] || *endptr != '\0') {
18         fprintf(stderr, "Error: el argumento debe ser un entero.\n");
19         return 2;
20     }
21
22     if (value < 0 || value > 255) {
23         fprintf(stderr, "Error: exit status valido entre 0 y 255.\n");
24         return 2;
25     }
26
27     return (int)value;
28 }
```

Justificación. Las Notas del curso describen `exit(code)` como la syscall que finaliza un proceso *limpiamente* y entrega el código al kernel, quien luego se lo pasa al padre vía `wait/waitpid`. Por convención 0 significa éxito y cualquier valor distinto, error. El kernel solo expone los **8 bits inferiores** del exit code en la variable `$?` del shell, lo que justifica la validación de rango `[0,255]`.

5.a Ejecutar y consultar el exit status

```
./myprog 7
echo $? # imprime 7
```

5.b Ejecutar otro comando si finaliza con éxito

```
./myprog 0 && echo "myprog termino con exito"
```

&& dispara el segundo comando solo si el primero retornó 0.

5.c Ejecutar otro comando si finaliza con error

```
./myprog 5 || echo "myprog termino con error"
```

|| dispara el segundo comando solo si el primero retornó un valor distinto de cero.

7. Ejercicio 6 — Versión propia de system()

La libc estándar provee `int system(const char *cmd)` como una abstracción sobre el patrón `fork()/exec()/wait()`. Una implementación equivalente es:

```
1 #include <errno.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 int mysystem(const char *command) {
6     pid_t pid;
7     int status;
8
9     if (command == NULL) {
10         return access("/bin/sh", X_OK) == 0;
11     }
12
13     pid = fork();
14     if (pid < 0) return -1;
15
16     if (pid == 0) {
17         execl("/bin/sh", "sh", "-c", command, (char *)NULL);
18         _exit(127); /* exec fall: convencin "comando no ejecutable" */
19     }
20
21     while (waitpid(pid, &status, 0) < 0) {
22         if (errno != EINTR) return -1;
23     }
24     return status;
25 }
```

Decisiones de diseño relevantes.

- `/bin/sh -c <cmd>` en lugar de `execvp`: permite que el comando contenga operadores de shell (pipes, redirecciones, &&, etc.).
- `_exit(127)` si `exec` falla: usa `_exit` y no `exit` para evitar que las funciones registradas con `atexit` y los buffers de `stdio` corran dos veces.
- **Reintento ante EINTR**: una señal puede interrumpir `waitpid` antes de que el hijo termine.

El `main` de prueba decodifica el `status` con `WIFEXITED/WEXITSTATUS` para reportar el exit code. Esta mecánica de dos pasos (`fork + exec`) es exactamente la que el shell usa para implementar redirecciones: el padre ajusta el ambiente del hijo (descriptores, variables) antes de invocar al nuevo programa.

8. Ejercicio 7 — Comunicación padre/hijo por pipe()

Idea. `pipe(p)` retorna dos descriptors: `p[0]` (lectura) y `p[1]` (escritura). Como `fork()` duplica los descriptors, ambos procesos los heredan; cada uno debe cerrar el extremo que no usa. Si el padre olvida cerrar el extremo de escritura, el hijo nunca verá EOF y se bloqueará indefinidamente.

```
1 int p[2];
2 pipe(p);
3 pid_t pid = fork();
4
5 if (pid == 0) {
6     /* Hijo: lee del pipe, imprime en stdout. */
7     close(p[1]);
8     char buffer[256];
9     ssize_t n;
10    printf("Hijo recibio: "); fflush(stdout);
11    while ((n = read(p[0], buffer, sizeof(buffer))) > 0)
12        write(STDOUT_FILENO, buffer, n);
13    close(p[0]);
14    _exit(0);
15 }
16
17 /* Padre: escribe el mensaje y cierra. */
18 close(p[0]);
19 write(p[1], msg, strlen(msg));
20 close(p[1]); /* dispara EOF en el hijo */
21 waitpid(pid, NULL, 0);
```

Compilación y ejecución:

```
gcc -Wall -Wextra -o pipe_padre_hijo pipe_padre_hijo.c
./pipe_padre_hijo "Hola desde el padre"
```

Salida:

```
Hijo recibio: Hola desde el padre
```

Limitación: un pipe sin nombre solo sirve entre procesos emparentados, porque los descriptors solo se heredan vía `fork`. Esa limitación se resuelve en el ejercicio siguiente.

9. Ejercicio 8 — Comunicación por FIFO

Un **FIFO** (*named pipe*) es un pipe con un nombre en el sistema de archivos. Aparece en `ls -l` con la marca `p` en los permisos. Se crea con la syscall `mkfifo(path, mode)` (o el comando equivalente) y se abre con la API estándar de archivos. Los datos *no se almacenan en disco*: solo el *nombre* vive en el filesystem; el buffer real está en el kernel, igual que un pipe común.

```
1 #define FIFO_PATH "/tmp/practico1_fifo_pipe"
2
3 mkfifo(FIFO_PATH, 0600);
4
5 if (fork() == 0) {
6     int fd = open(FIFO_PATH, O_RDONLY);
7     char buf[256]; ssize_t n;
8     printf("Hijo recibio: ");
9     while ((n = read(fd, buf, sizeof(buf))) > 0)
```

```

10     write(STDOUT_FILENO, buf, n);
11     close(fd); _exit(0);
12 }
13
14 int fd = open(FIFO_PATH, O_WRONLY);
15 write(fd, msg, strlen(msg));
16 close(fd);
17 wait(NULL);
18 unlink(FIFO_PATH);

```

Sincronización implícita. `open(fifo, O_RDONLY)` se bloquea hasta que algún proceso abra el otro extremo (`O_WRONLY`), y viceversa. Esta propiedad permite coordinar procesos que se conocen únicamente por la ruta del FIFO.

Limpieza. El FIFO sobrevive al cierre de ambos extremos; hay que removerlo explícitamente con `unlink()`.

10. Ejercicio 9 — Mini shell minish

El programa `minish.c` implementa un mini shell que lee comandos en bucle hasta recibir `Ctrl-D` (EOF en `stdin`) y reconoce los siguientes operadores binarios:

Operador	Mecanismo
<code>cmd1 ; cmd2</code>	Ejecutar <code>cmd1</code> , luego <code>cmd2</code> .
<code>cmd1 && cmd2</code>	Ejecutar <code>cmd2</code> solo si <code>cmd1</code> retornó 0.
<code>cmd1 cmd2</code>	Ejecutar <code>cmd2</code> solo si <code>cmd1</code> retornó <code>!= 0</code> .
<code>cmd1 & cmd2</code>	<code>cmd1</code> en background, <code>cmd2</code> en foreground.
<code>cmd1 cmd2</code>	<code>pipe(p)</code> + dos <code>fork/exec</code> + <code>dup2</code> .
<code>cmd < archivo</code>	<code>open(archivo, O_RDONLY) + dup2(fd, 0)</code> .
<code>cmd > archivo</code>	<code>open(..., O_WRONLY O_CREAT O_TRUNC) + dup2(fd, 1)</code> .
<code>cmd » archivo</code>	<code>open(..., O_WRONLY O_CREAT O_APPEND) + dup2(fd, 1)</code> .

Cuadro 2: Operadores reconocidos por `minish` y su mecanismo POSIX.

Estructura.

1. **Loop principal** (`getline()`). Si retorna `< 0` (EOF) el shell sale.
2. **Tokenización** por espacios y tabulaciones; los tokens se almacenan en un arreglo de punteros al buffer original modificado in-situ con `\0`.
3. **Despacho recursivo** (`dispatch`). Busca operadores en orden de prioridad baja a alta y descompone el comando en (`left`, `op`, `right`) llamando recursivamente a cada lado.
4. **Comando simple**: `fork` + (opcional `dup2`) + `execvp`; el padre llama a `waitpid` salvo en background.
5. **Pipe**: dos `fork` con un pipe previo; el padre cierra ambos extremos para que los hijos vean EOF.

Salida esperada.

```

$ ./minish
minish$ echo hola ; echo mundo
hola
mundo

```

```

minish$ sleep 1 & echo "sigo en foreground"
[bg] pid=12345
sigo en foreground
minish$ true && echo "se ejecuto AND"
se ejecuto AND
minish$ false || echo "se ejecuto OR"
se ejecuto OR
minish$ echo hola | tr a-z A-Z
HOLA
minish$ ^D

```

Limitaciones. El parsing es deliberadamente simple: no soporta comillas, expansión de variables ni globbing. El propósito didáctico es ilustrar el modelo `fork/exec/wait/pipe/dup2`, no reimplementar `bash`. La implementación completa se encuentra en `resolucion-9/minish.c`.

11. Ejercicio 10 — Captura de SIGINT

Marco teórico. Las Notas del curso describen las señales como un mecanismo básico de comunicación entre procesos para notificar eventos. `SIGINT` (interrupción) es la señal por defecto que el shell envía al proceso de *foreground* cuando el usuario presiona `Ctrl-C`. Sin manejador, el kernel termina el proceso. Este ejercicio invierte ese comportamiento preguntando antes de finalizar.

```

1 #include <ctype.h>
2 #include <errno.h>
3 #include <signal.h>
4 #include <stdio.h>
5 #include <string.h>
6 #include <unistd.h>
7
8 static void sigint_handler(int sig) {
9     const char prompt[] =
10         "\nSIGINT recibida. Desea finalizar el proceso? [s/N]: ";
11     char answer[64];
12     ssize_t nread;
13     (void)sig;
14
15     write(STDOUT_FILENO, prompt, sizeof(prompt) - 1);
16     do {
17         nread = read(STDIN_FILENO, answer, sizeof(answer));
18     } while (nread < 0 && errno == EINTR);
19
20     for (ssize_t i = 0; i < nread; i++) {
21         unsigned char c = (unsigned char)answer[i];
22         if (!isspace(c)) {
23             if (c == 's' || c == 'S' || c == 'y' || c == 'Y')
24                 _exit(0);
25             _exit(-1);
26         }
27     }
28     _exit(-1);
29 }
30
31 int main(void) {
32     struct sigaction sa = {0};
33     sa.sa_handler = sigint_handler;
34     sigemptyset(&sa.sa_mask);
35     sa.sa_flags = SA_RESTART;

```

```

36     sigaction(SIGINT, &sa, NULL);
37
38     printf("Proceso en ejecucion. PID=%d\n", (int)getpid());
39     for (;;) pause();
40 }

```

Funciones async-signal-safe. Las funciones de `<stdio.h>` no son seguras para invocar dentro de un handler porque usan buffers y mutexes internos. POSIX define un subconjunto reducido de funciones *async-signal-safe* (`read`, `write`, `_exit`, `sigaction`, etc.) y este código las usa exclusivamente.

Detalle del exit -1. El kernel solo expone los 8 bits inferiores en `$_?`, así que `_exit(-1)` se observa como 255. El programa retorna lógicamente -1; la representación en el shell es un detalle de codificación.

12. Ejercicio 11 — Mínimo global con p hijos y un pipe

Esquema. El padre crea `p` procesos hijos. Cada hijo recibe un bloque de `N/p` elementos (la asignación se hace por simple aritmética `[i*chunk, (i+1)*chunk)`), calcula su *mínimo local* y lo envía al padre por un pipe único compartido. El padre lee `p` mensajes, va aplicando `min`, y reporta el resultado tras esperar a todos los hijos con `waitpid`.

```

1  typedef struct {
2      int child_id;
3      int local_min;
4  } ChildResult;
5
6  /* ... validaciones de p y N ... */
7  pipe(fd);
8  for (int i = 0; i < p; i++) {
9      pid_t pid = fork();
10     if (pid == 0) {
11         int start = i * chunk, end = start + chunk;
12         int local_min = arr[start];
13         for (int k = start + 1; k < end; k++)
14             if (arr[k] < local_min) local_min = arr[k];
15
16         ChildResult r = { i, local_min };
17         close(fd[0]);
18         write_full(fd[1], &r, sizeof r);
19         close(fd[1]);
20         _exit(0);
21     }
22     children[i] = pid;
23 }
24 close(fd[1]);
25
26 int global_min = INT_MAX;
27 for (int i = 0; i < p; i++) {
28     ChildResult r;
29     read_full(fd[0], &r, sizeof r);
30     if (r.local_min < global_min) global_min = r.local_min;
31 }
32 close(fd[0]);
33 for (int i = 0; i < p; i++) waitpid(children[i], NULL, 0);

```

¿Por qué un solo pipe basta? POSIX garantiza que escrituras de hasta PIPE_BUF bytes son **atómicas** en un pipe: dos escritores nunca mezclan sus mensajes. Como `sizeof(ChildResult)` es muy pequeño (<16 bytes), el padre puede leer mensajes completos en cualquier orden sin corrupción.

Ejemplo.

```
./minimo_global_pipe 3 8 4 7 10 -2 5
Hijo 0 -> minimo local = 4
Hijo 1 -> minimo local = 7
Hijo 2 -> minimo local = -2
Minimo global = -2
```

13. Ejercicio 12 — Lectura con timeout usando alarm

Marco teórico. Las Notas del curso muestran (Listado 6) cómo implementar timeouts: el kernel envía la señal `SIGALRM` cuando vence el tiempo programado con `alarm(N)`. El handler asociado puede marcar una bandera y, al volver de la syscall bloqueante, el programa decide si retornó por dato disponible o por interrupción.

```
1 static volatile sig_atomic_t timed_out = 0;
2
3 static void alarm_handler(int sig) {
4     (void)sig;
5     timed_out = 1;
6 }
7
8 int main(int argc, char *argv[]) {
9     int N = atoi(argv[1]);
10    char buf[256];
11
12    struct sigaction sa = {0};
13    sa.sa_handler = alarm_handler;
14    sigemptyset(&sa.sa_mask);
15    sa.sa_flags = 0; /* sin SA_RESTART */
16    sigaction(SIGALRM, &sa, NULL);
17
18    alarm(N);
19    ssize_t n = read(STDIN_FILENO, buf, sizeof(buf) - 1);
20    alarm(0);
21
22    if (n > 0) { buf[n] = 0; printf("Dato: %s", buf); return 0; }
23    if (n == 0) { puts("EOF"); return 0; }
24    if (errno == EINTR && timed_out) {
25        printf("Timeout: sin datos en %d s\n", N);
26        return 124;
27    }
28    perror("read");
29    return 1;
30 }
```

Por qué sin SA_RESTART. Si el handler se instalara con `SA_RESTART`, el kernel reanudaría automáticamente la syscall `read` después de la señal y nunca veríamos `EINTR`. El timeout sería invisible. Sin `SA_RESTART`, `read` retorna `-1` con `errno = EINTR` y podemos detectar la condición consultando la bandera `timed_out`.

Por qué volatile sig_atomic_t. volatile impide que el compilador optimice la lectura asumiendo que la variable no cambia entre dos accesos. sig_atomic_t es el tipo más amplio que el sistema garantiza poder leer y escribir atómicamente desde un handler.

Por qué 124 como exit status. coreutils usa 124 como convención para indicar finalización por timeout, igual que el comando timeout(1). No es mágico, pero facilita scripts que distinguen este caso de otros errores.

Pruebas.

```
(sleep 1; echo "hola") | ./lee_con_timeout 5  
Dato recibido: hola
```

```
(sleep 3; echo "tarde") | ./lee_con_timeout 1  
Timeout: no se recibieron datos en 1 segundos.  
echo $? # imprime 124
```

Cierre

A lo largo del práctico se ejercitó la API POSIX como manifestación concreta de los conceptos teóricos del SO: el modelo de procesos (fork/exec/wait), la abstracción de archivos y descriptores (open/read/write/dup2), los mecanismos de comunicación (pipe, FIFO) y de notificación asíncrona (signal, alarm). Los operadores del shell funcionan exactamente como combinaciones de esas syscalls; el mini shell del ejercicio 9 hace explícita esa correspondencia. La materia en los prácticos siguientes profundiza en los temas de planificación de CPU, gestión de memoria y sistemas de archivos que se asoman aquí en su forma más cruda.

Referencias

1. Marcelo Arroyo. *Notas del curso de Sistemas Operativos*, capítulos 1, 2 y 3. UNRC, 2026. <https://marceloarroyo.gitlab.io/cursos/SO/>
2. Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, capítulo 1, 5ª edición. Pearson, 2024.
3. POSIX.1-2017 / Single UNIX Specification. The Open Group, 2018.
4. Sección 2 de man (manuales de las syscalls): man 2 fork, man 2 execve, man 2 pipe, man 2 sigaction, etc.